Project Documentation Report

Smart library request workflow

Date	8 April 2026
Team ID	SWTID-2026-2921
Project Name	Smart library request workflow
Maximum Marks	2 Marks

Team Members

S.No	Name	RegNo
1	KAVIYARASAN R	812823205038
2	ROHINI M	812823205059
3	KALAIVANI S	812823205030
4	KAVIN VB	812823205037
5	JASMEERAN N	812823205025

1. INTRODUCTION:

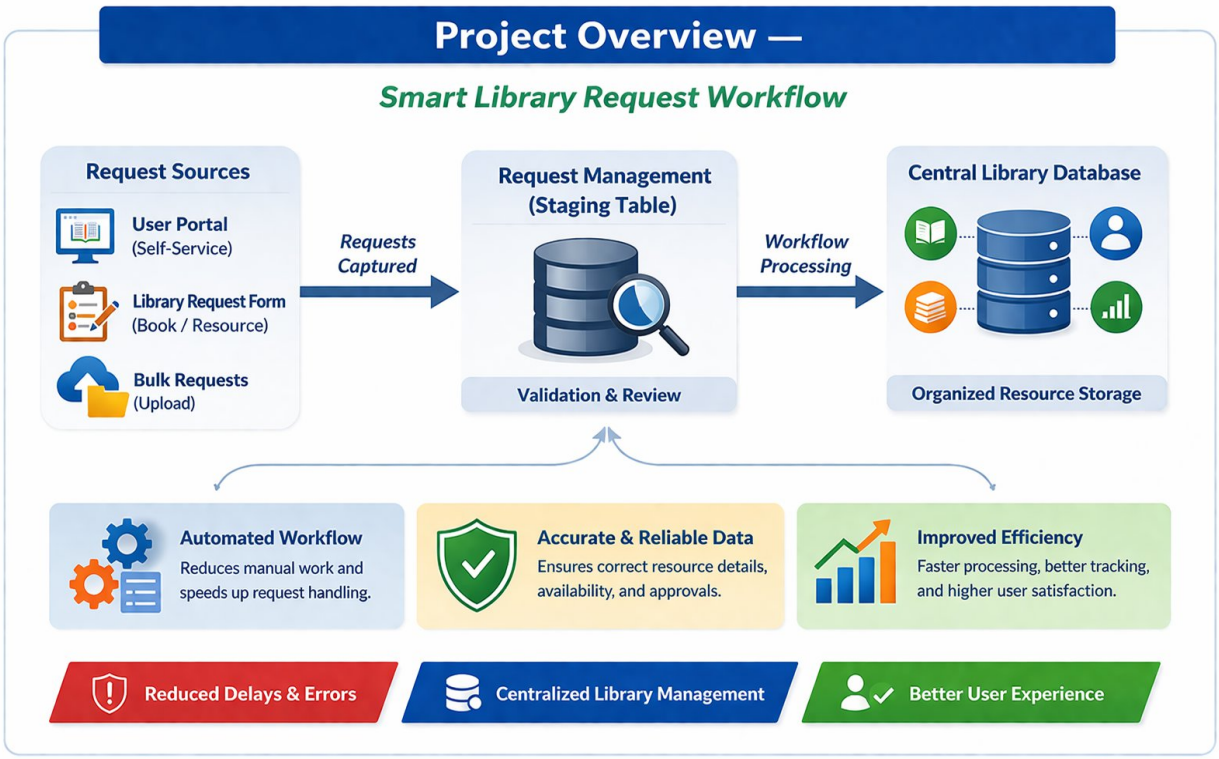
Project Purpose:

The purpose of the Smart Library Request Workflow is to automate and simplify the process of requesting, approving, and managing library resources using ServiceNow. It aims to reduce manual effort, eliminate errors, speed up request handling, and ensure efficient tracking and availability of library materials while improving user experience and productivity.

Project Overview:

The Smart Library Request Workflow is an automated solution developed using ServiceNow to streamline the process of requesting, approving, and managing library resources. The system replaces traditional manual methods with a centralized digital platform, allowing users to easily submit requests for books or other materials through a self-service portal.

Once a request is submitted, the system automatically validates the data, checks resource availability, and routes the request for approval if required. Upon approval, tasks are generated for issuing or procuring the requested items. Throughout the process, users receive real-time notifications about the status of their requests.



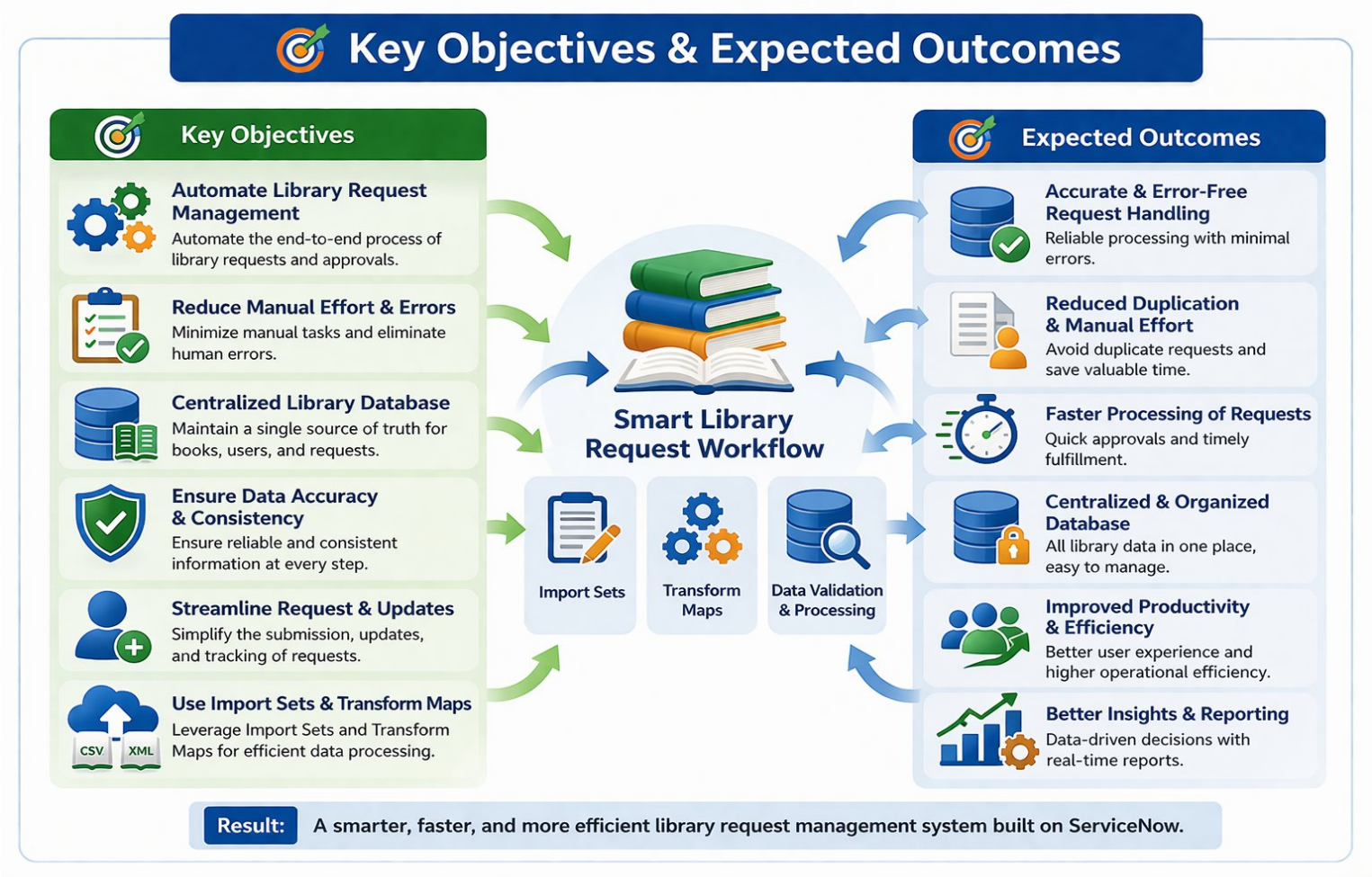
Key Objectives & Expected Outcomes:

Key Objectives:

The key objectives of this project are to automate the management of library requests using ServiceNow and reduce dependency on manual processes. The project focuses on creating a centralized system for handling book and resource requests while ensuring data accuracy, consistency, and proper tracking. It also aims to streamline request submission, validation, approval, and fulfillment by using automated workflows, thereby improving efficiency and user experience.

Expected Outcomes:

The expected outcomes of this project include accurate and error-free handling of library requests with reduced duplication and minimal manual effort. The system enables faster processing of requests and provides a centralized, well-organized database for managing library resources. It also improves productivity and operational efficiency by ensuring easy access to up-to-date and reliable information on books, requests, and availability using ServiceNow.

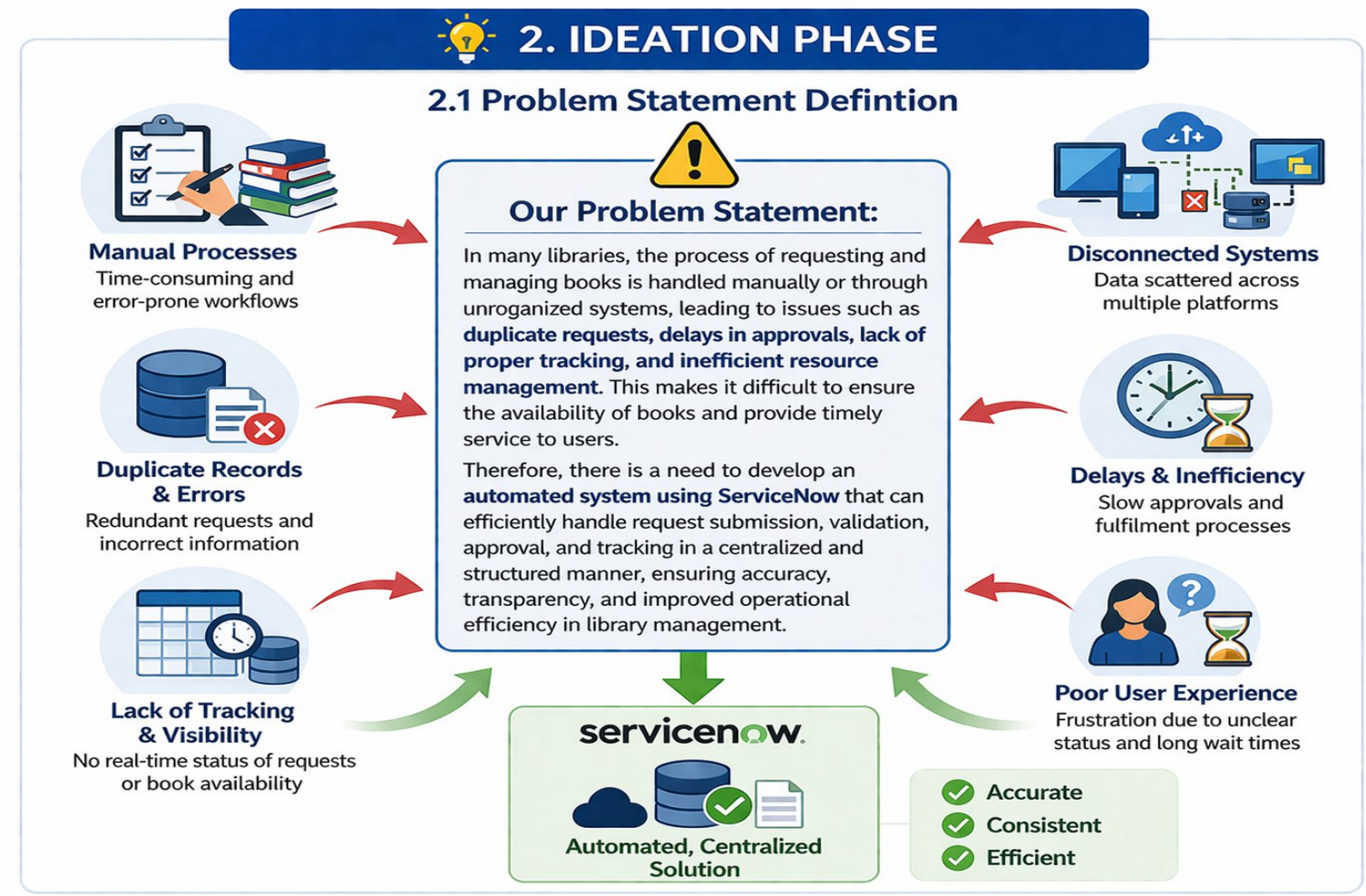


2. IDEATION PHASE:

Problem Statement Definition

Our Problem Statement:

In many libraries, the process of requesting and managing books is handled manually or through unorganized systems, leading to issues such as duplicate requests, delays in approvals, lack of proper tracking, and inefficient resource management. This makes it difficult to ensure the availability of books and provide timely service to users.



Problem Statements Identified

PS-1

Problem Statement (PS): PS-1
I am (Customer): A librarian or library system administrator
I'm trying to: manage and process book/resource requests submitted by users through the system
But: the current process is manual, time-consuming, and lacks proper tracking, leading to duplicate requests and delays in approvals
Because: there is no automated workflow to validate, approve, and manage requests efficiently
Which makes me feel: frustrated and inefficient due to repetitive manual work and lack of system transparency

PS-2

I am (Customer): A librarian or library system administrator
I'm trying to: manage and process book/resource requests submitted by users
But: the current process is manual, time-consuming, and lacks proper tracking, leading to duplicate requests and delays in approvals
Because: there is no automated workflow to efficiently validate, approve, and manage requests
Which makes me feel: frustrated and inefficient due to repetitive manual tasks and lack of system transparency

PS-3

I am (Customer): A library management team member / organization user
I'm trying to: maintain a consistent and reliable database of library resources and requests
But: the system contains duplicate requests, outdated records, and inconsistent data
Because: there is no proper validation and automated workflow for managing library data
Which makes me feel: concerned about data accuracy, tracking, and effective decision-making

Empathy Map:

Empathy Mapping

User: University Researcher / Student

SAYS

- Finding the right resource is very time-consuming.
- Requesting books or materials is too manual and slow.
- Checking availability across different branches is confusing.
- Reserving a study space or equipment is a nightmare.
- I need a central place to view my digital requests.

THINKS

- There should be a **smart, digital workflow** for all requests.
- My time is better spent on research, not on system navigation.
- The system should know my borrowing history and make suggestions.
- An integrated, accurate catalog would be much more effective.
- It would be great if digital and physical resources were linked.

SMART LIBRARY SYSTEM

INTEGRATED WORKFLOW

SEES

- Multiple disparate systems (catalog, study rooms, digital library).
- Duplication of data and search efforts across platforms.
- Slow response times and outdated resource availability status.
- Conflicting information from different library websites.
- A lot of paperwork and email confirmations for simple requests.

HEARS

- Complaints from colleagues about system user-friendliness.
- Direct request to a librarian about status updates.
- Faculty requests for data to support grants.
- General conversation among peers about new digital resources.

PAIN POINTS

- Slow and complicated manual request processes.
- High risk of errors in data and inventory.
- Difficulty managing different types of data (books, journals, digital).
- Lack of real-time system integration.
- Time wasted during onboarding for new users.

GAINS

- Automated and faster workflow for all requests.
- High accuracy and data consistency.
- Easy access to a central hub for digital resources.
- Real-time tracking of resource availability and usage.
- Onboarding is smooth and efficient.

Brainstorming and Idea Prioritization:

1. Problem Statement

The project focuses on automating employee data import into ServiceNow to reduce manual work, minimize errors, and maintain accurate records. It also includes designing a smart library request workflow to streamline request handling and improve efficiency.

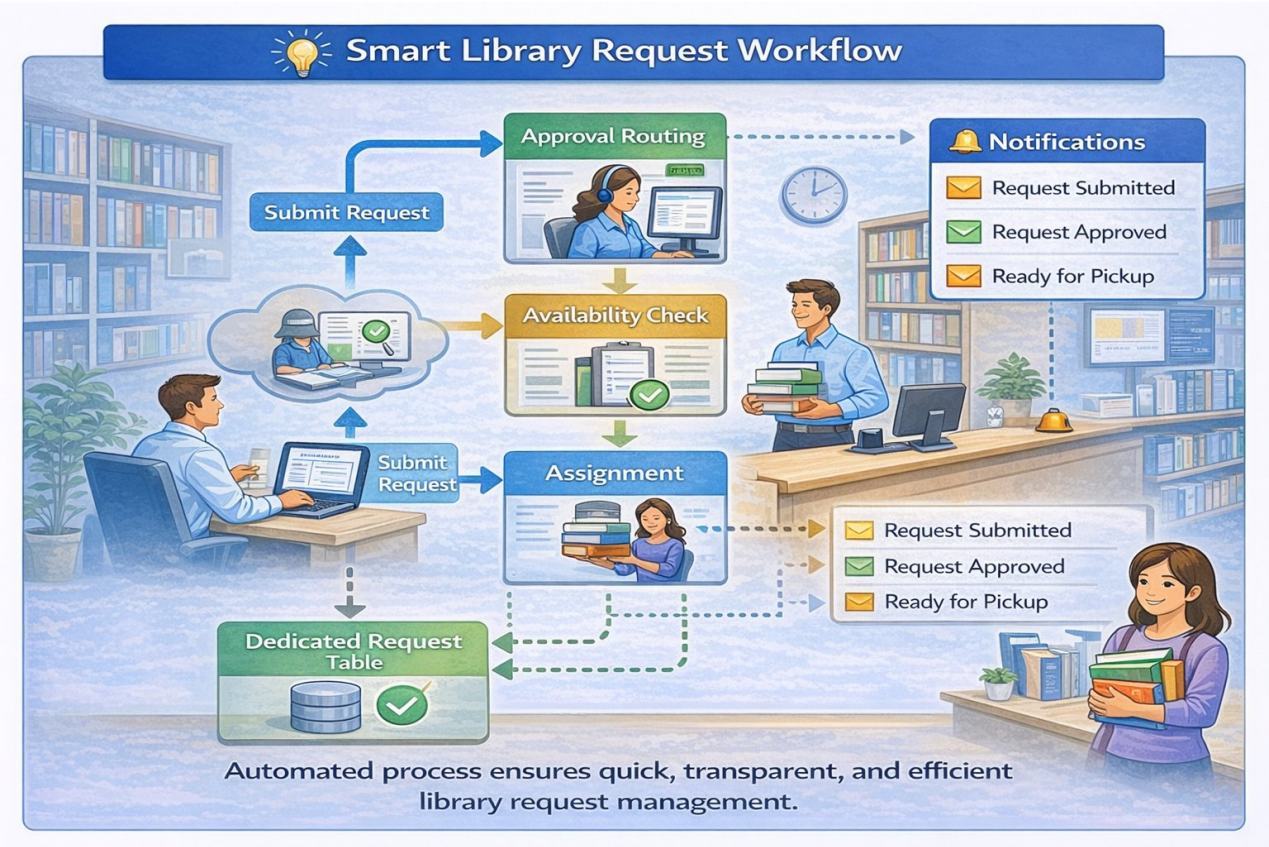
2. Brainstorming

- For the smart library request workflow, requests can be submitted through forms or APIs, processed using Flow Designer or Business Rules, and stored in a custom or existing table.
- For employee data automation, import methods include Scheduled Import Sets, REST API, or Manual Upload, with processing via Transform Maps or Business Rules, and storage in the sys_user or a Custom Table.

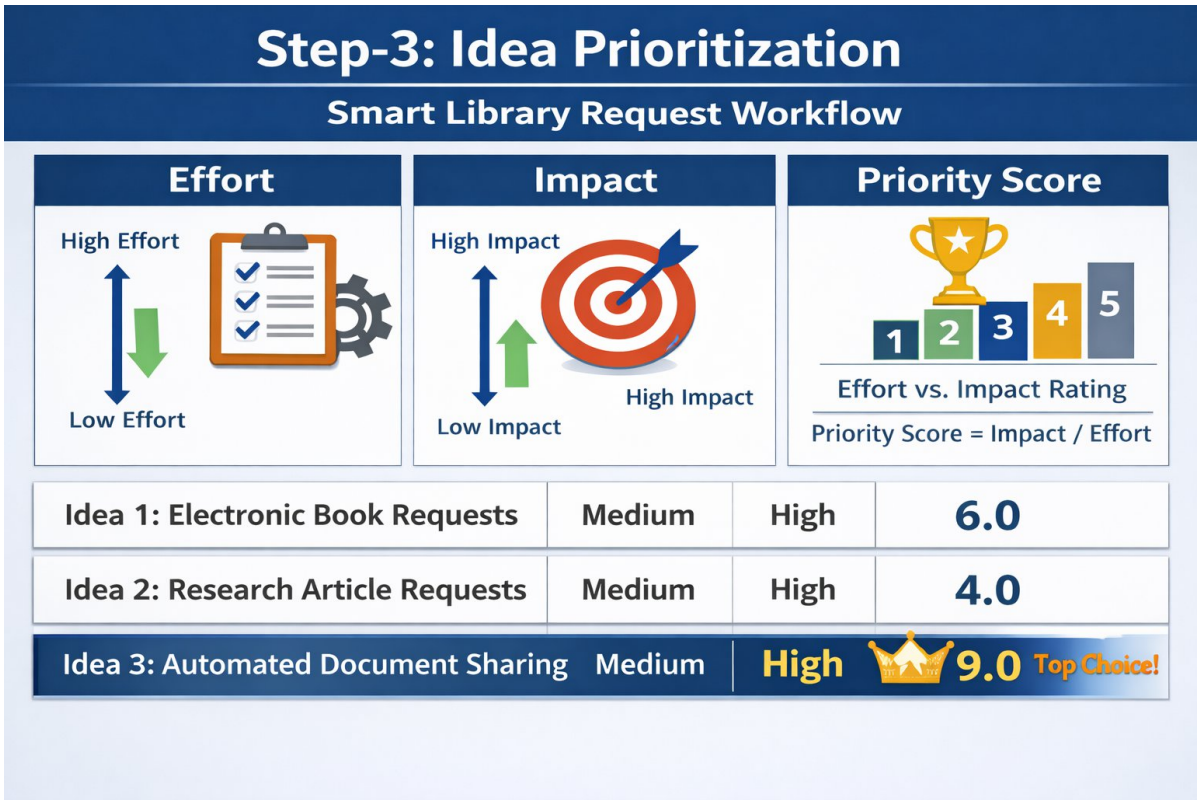
3. Prioritization

The selected approach is **Import Sets + Transform Maps + sys_user Table**. This combination is chosen for its better performance, higher accuracy in data handling, and ease of implementation within the system.

Step-1: Team Gathering, Collaboration and Select the Problem statement.

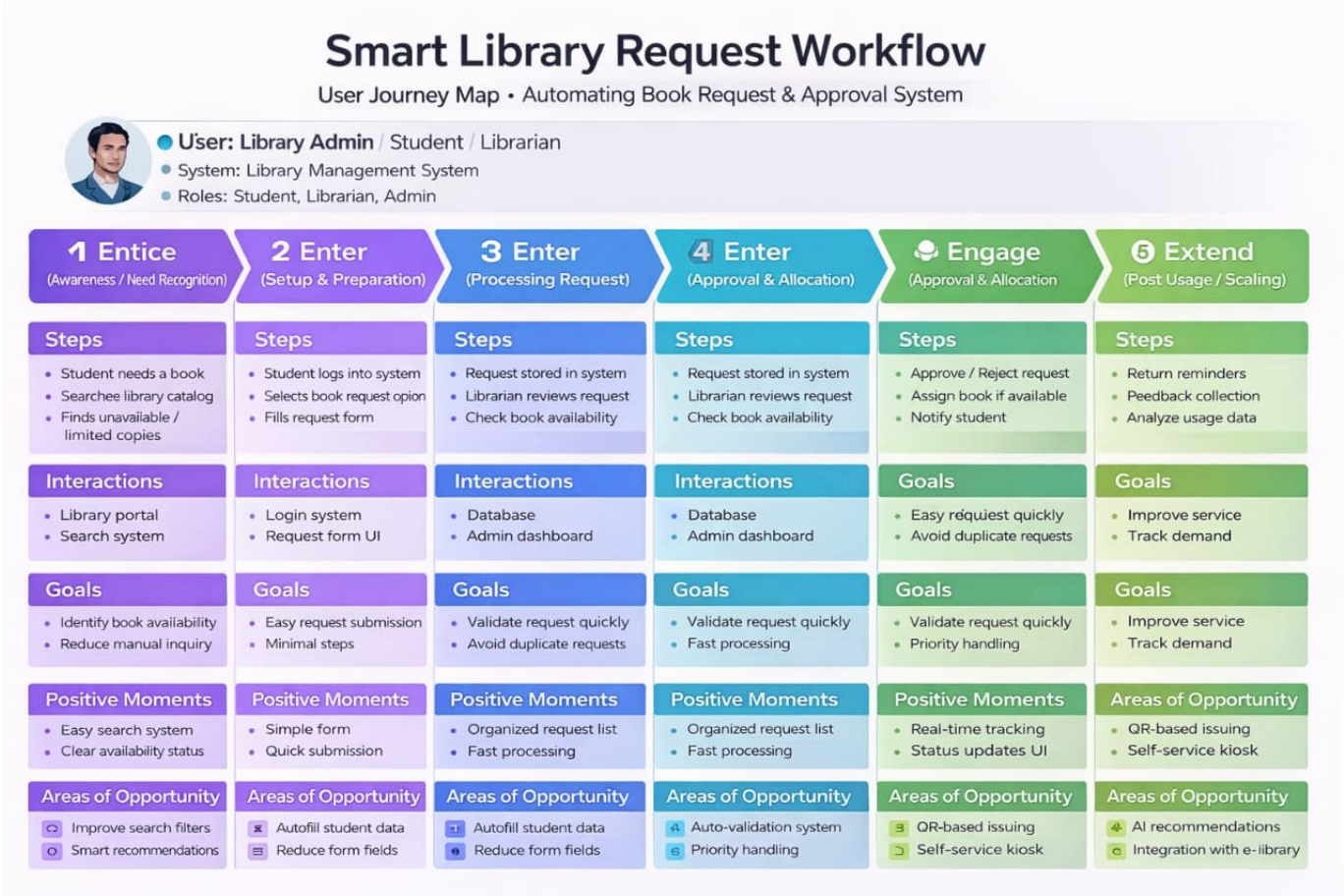


Step-2: Brainstorm, Idea Listing and Grouping



3. REQUIREMENT ANALYSIS:

3.1 Customer Journey Map



Solution Requirements:

Functional Requirements:

Following are the functional requirements of the proposed solution.

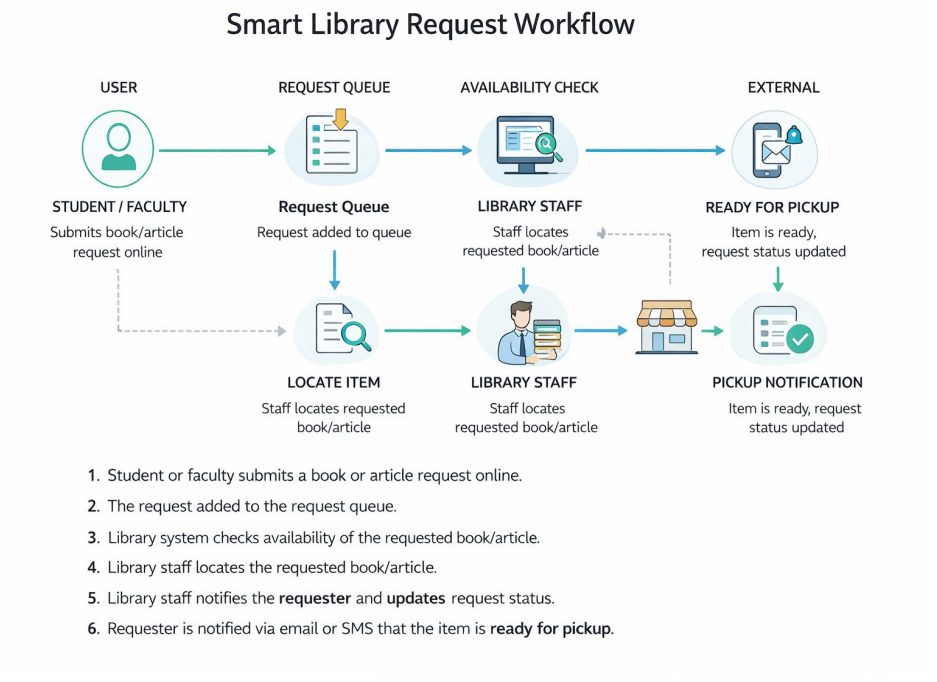
FR No.	Functional Requirement	Sub Requirement
FR-1	Book Request Import	Upload book request data through CSV/XLSX file Automatic creation of import table (import_library_request)
FR-2	Data Transformation & Mapping	Create Transform Map between import_library_request and library_request table Map basic fields (Book Title, Author, Category)
FR-3	Data Validation & Processing	Execute Transform Map to insert records Validate correct data insertion into library_request table
FR-4	Incident Form Automation	Display user details on Library Request form Show Department using dot-walking.
FR-5	Dynamic Data Display	Auto-update fields when "Requested By" user changes
FR-6	Security & Access Control	Restrict request import access to authorized users Restrict approval access to Librarian role.Ensure data visibility based on roles (ACLs)

Non-functional Requirements:

FR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	The system should provide a user-friendly interface for submitting and tracking library requests. Users should be able to request books easily, and the request status should be displayed clearly without additional navigation..
NFR-2	**Security**	Only authorized users (Librarian/Admin roles) should be allowed to approve or manage requests. Data visibility must follow Role-Based Access Control (RBAC) and Access Control Lists (ACLs).
NFR-3	**Reliability**	The system should handle invalid or incomplete request data gracefully. Errors during request submission or processing should be logged without affecting other requests.
NFR-4	**Performance**	The system should process library requests quickly without affecting platform performance. Request forms and status updates should load instantly without delay.
NFR-5	**Availability**	The system should be available at all times with minimal downtime. Request submission, approval workflow, and tracking features should function consistently.
NFR-6	**Scalability**	The system should support both small and large numbers of requests (e.g., hundreds or thousands of requests) without performance degradation and scale efficiently as usage increases.

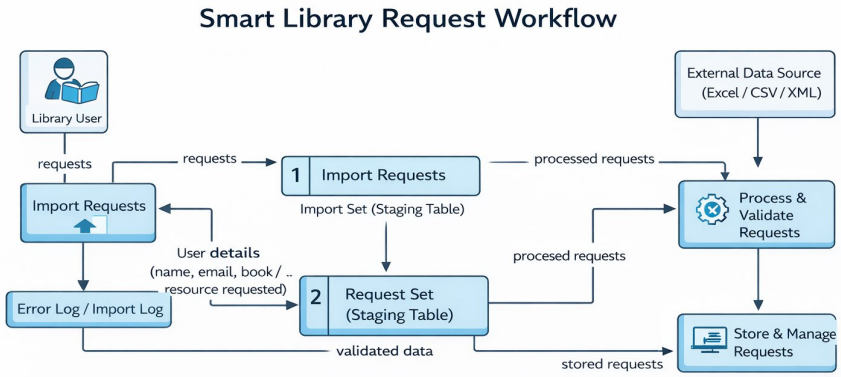
3.3 Data Flow Diagrams:

Level 1 DFD



1. Student or faculty submits a book or article request online.
2. The request added to the request queue.
3. Library system checks availability of the requested book/article.
4. Library staff locates the requested book/article.
5. Library staff notifies the **requester** and **updates** request status.
6. Requester is notified via email or SMS that the item is **ready for pickup**.

Level 2 DFD



1. User submits request (book/resource) via web form to **Smart Library**.
2. Request Set loads the data into a staging table.
3. Data is validated and processed using **Transform Maps**.
4. Transformed data is stored in the request table (sys_user_request).
5. Requests are displayed and used by Smart Library applications.

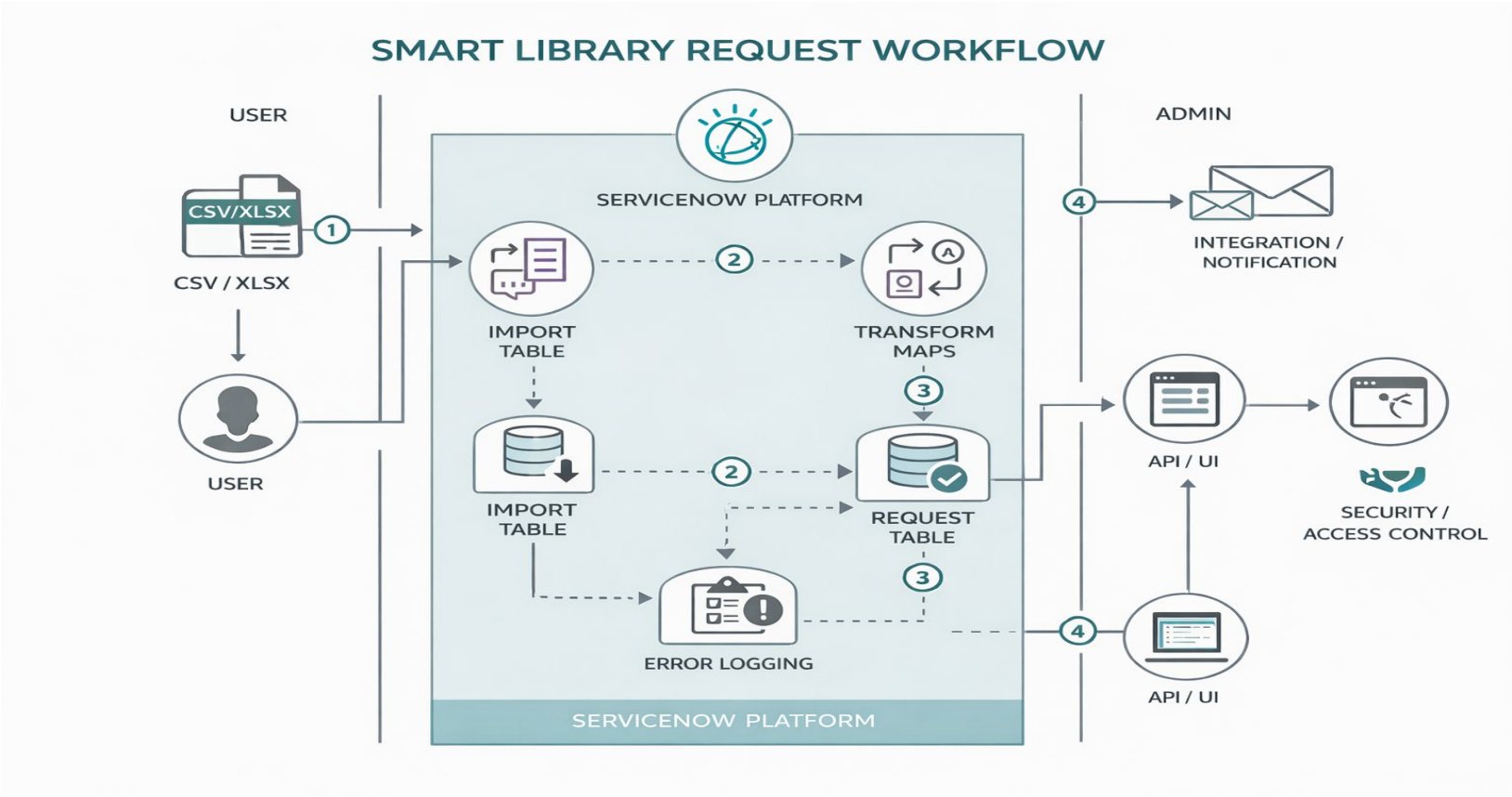
User Stories:

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria
Service Now Admin	Data Import Setup	USN-1	As a Service Now Admin, I want to upload employee data (CSV/XLSX) so that it can be stored in the system	File is uploaded successfully and data appears in import table
ServiceNow Admin	Request Import Setup	USN-2	As an Admin, I want the system to automatically create an import table (`import_library_request`)	import table is created with all uploaded records
ServiceNow Admin	Request Transformation	USN-3	Create a Transform Map to move data to request table	Transform map is created and linked to correct tables
ServiceNow Admin	Request Transformation	USN-4	Map fields (Requester Name, Email, Book Title, Author, Request Type, Department)	All fields are correctly mapped without errors
ServiceNow Admin	Request Validation	USN-5	Validate transformed data for accuracy	Data is correctly inserted into system tables (e.g., user/request tables)
ServiceNow Admin	Error Handling	USN-6	Log errors for missing or incorrect data	Errors are logged without stopping the process
ServiceNow Admin	Form Configuration	USN-7	Configure forms to display employee and request details	Fields are added and visible on form

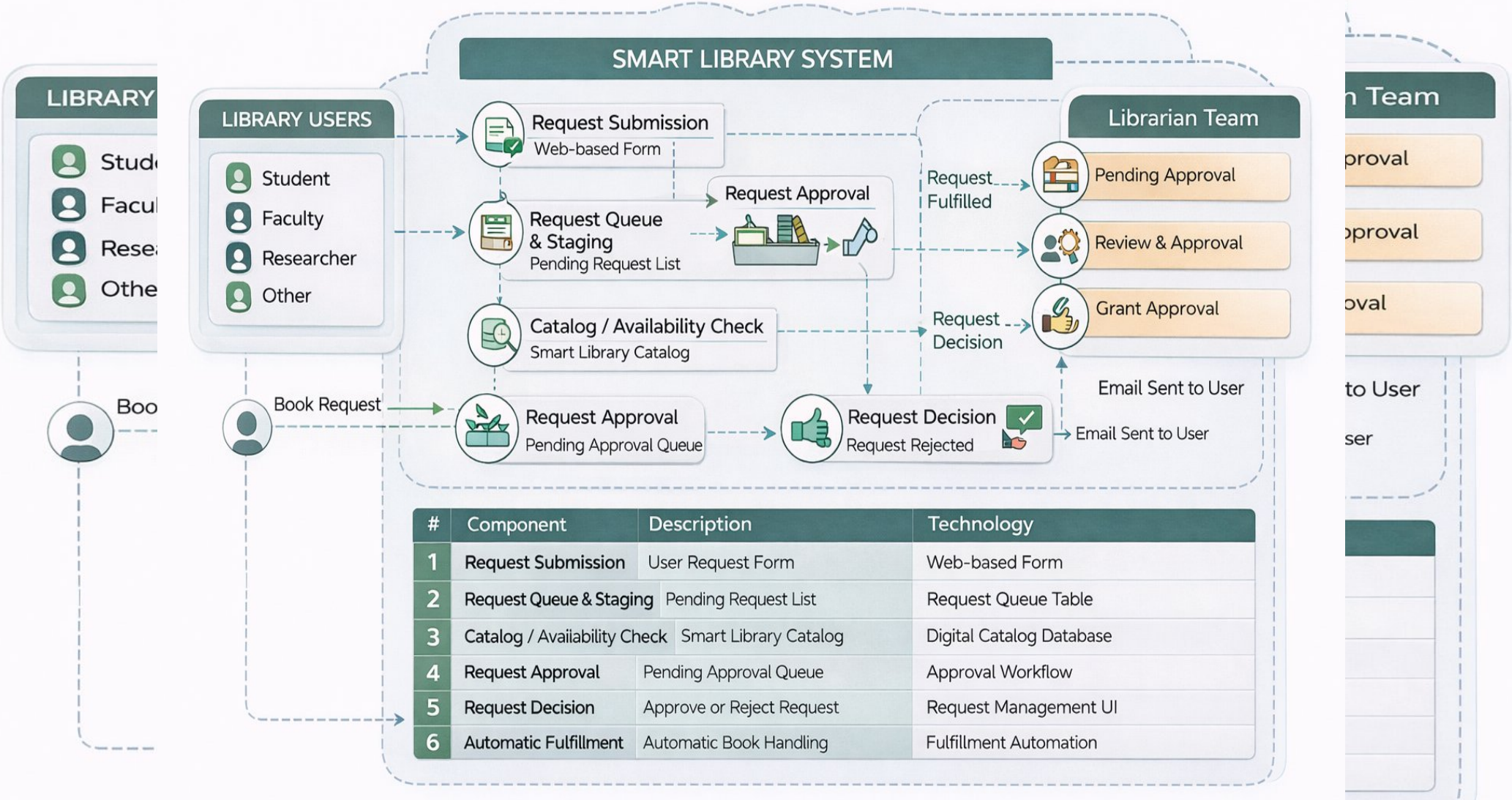
Technology Stack:

Technical Architecture:

The **Smart Library Request Workflow** uses ServiceNow to automate data import, transformation, validation, and request management through forms, Import Sets, and Transform Maps.



Smart Library Request Workflow –
Automating Book Requests & Approval Process



#	Component	Description	Technology
1	Request Submission	User Request Form	Web-based Form
2	Request Queue & Staging	Pending Request List	Request Queue Table
3	Catalog / Availability Check	Smart Library Catalog	Digital Catalog Database
4	Request Approval	Pending Approval Queue	Approval Workflow
5	Request Decision	Approve or Reject Request	Request Management UI
6	Automatic Fulfillment	Automatic Book Handling	Fulfillment Automation

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	User-facing portal for students, faculty, and researchers to request books, track request status, and view availability. Admin dashboard for librarians to manage requests, approvals, and inventory.	HTML5, CSS3, JavaScript, React JS, Bootstrap
2	Application Logic-1	Core business logic for handling book requests, approvals, request prioritization, and request validation.	Python, Flask / Java Spring Boot
3	Application Logic-2	Voice-based request submission allowing users to search and request books using speech commands.	IBM Watson Speech to Text
4	Application Logic-3	AI assistant to answer user queries related to book availability, due dates, fines, and request status.	IBM Watson Assistant
5	Database	Stores book details, user information, request history, approval status, and borrowing records.	MySQL / PostgreSQL
6	Cloud Database	Centralized cloud storage for library data access across departments and backup management.	IBM Db2 on Cloud / IBM Cloudant
7	File Storage	Stores digital resources such as e-books, research papers, user documents, and library reports.	IBM Cloud Object Storage / Local File System
8	External API-1	Sends notifications to users for request approval, availability updates, due reminders, and return alerts.	Twilio API, SendGrid API
9	External API-2	Integration with external library catalogs or academic databases for checking book availability.	Google Books API / Open Library API
10	Machine Learning Model	Predicts book demand, recommends books to users, and analyzes borrowing patterns.	IBM Watson Machine Learning, Scikit-learn
11	Infrastructure (Server / Cloud)	Cloud deployment for hosting library system, database, AI modules, and request workflow services.	IBM Cloud, Docker, Kubernetes, Cloud Foundry

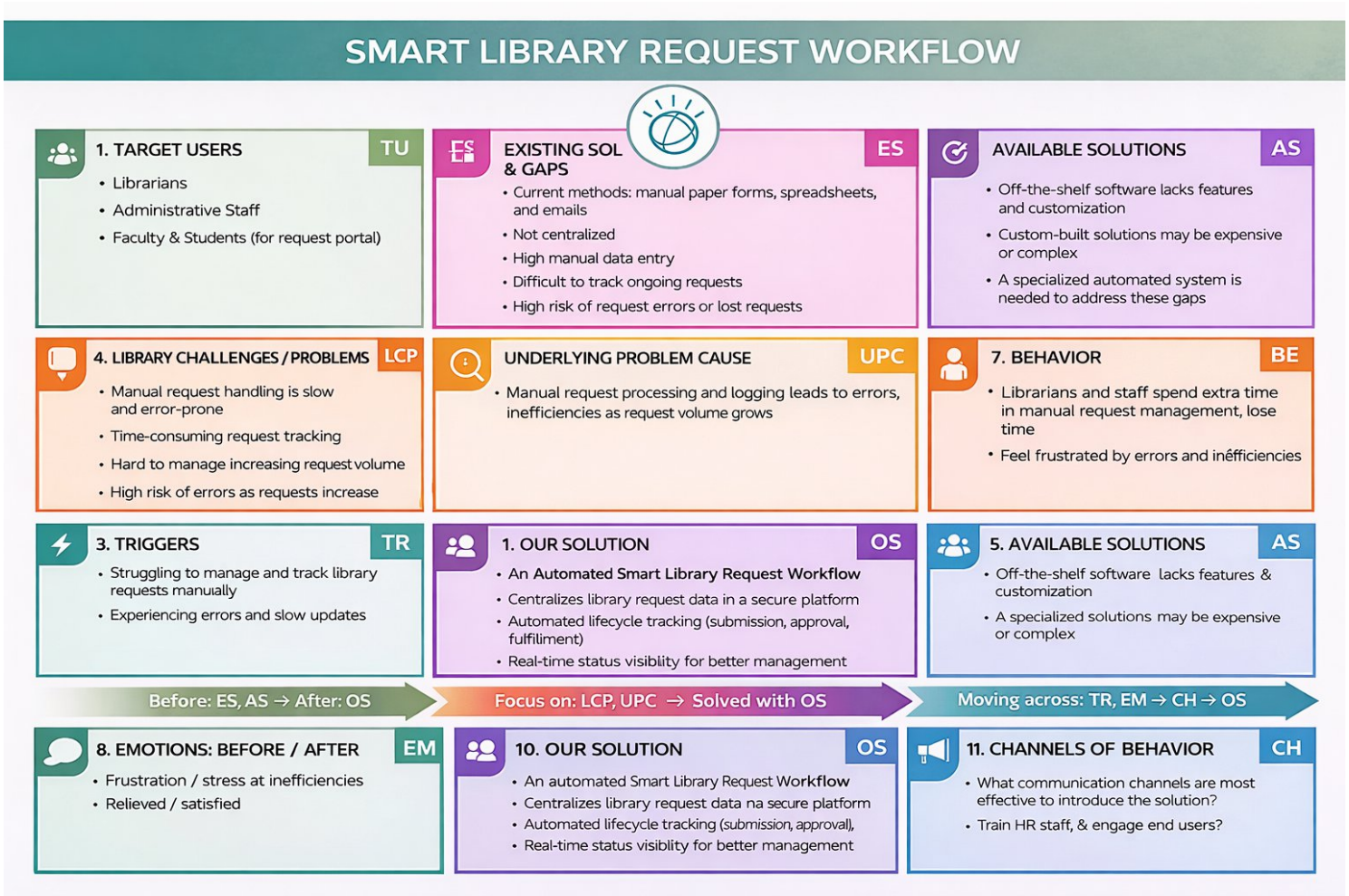
Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source frameworks are used for developing the frontend, backend, APIs, and AI-based recommendation modules of the smart library request workflow system.	React JS, Flask, Spring Boot, Node.js, Scikit-learn
2	Security Implementations	Secure login and role-based access for students, faculty, and librarians. User credentials and request data are protected during transmission and storage.	SHA-256, AES Encryption, HTTPS, JWT Authentication, IBM IAM, OWASP Security Standards
3	Scalable Architecture	The system follows a 3-tier architecture including presentation, application, and database layers, allowing independent scaling for request handling and catalog management.	Microservices Architecture, Docker, Kubernetes
4	Availability	The smart library system ensures continuous availability using cloud infrastructure, backup databases, and distributed servers for handling multiple user requests.	Load Balancer, IBM Cloud Availability Zones, Distributed Servers
5	Performance	Fast response time for book search, request processing, and catalog browsing using caching and optimized queries.	Redis Cache, CDN, Optimized SQL Queries
6	AI-Based Recommendation	The system recommends books to users based on borrowing history, search patterns, and popular demand.	IBM Watson Machine Learning, Scikit-learn, Recommendation Algorithms

4. PROJECT DESIGN

Problem – Solution Fit:

Manual handling of library requests can lead to errors, delays, and difficulty in tracking records as the number of requests increases. The **Smart Library Request Workflow** automates the entire process by centralizing data, reducing manual effort, and ensuring accurate request handling. This improves efficiency, saves time, and provides better visibility and management of library requests within the system.



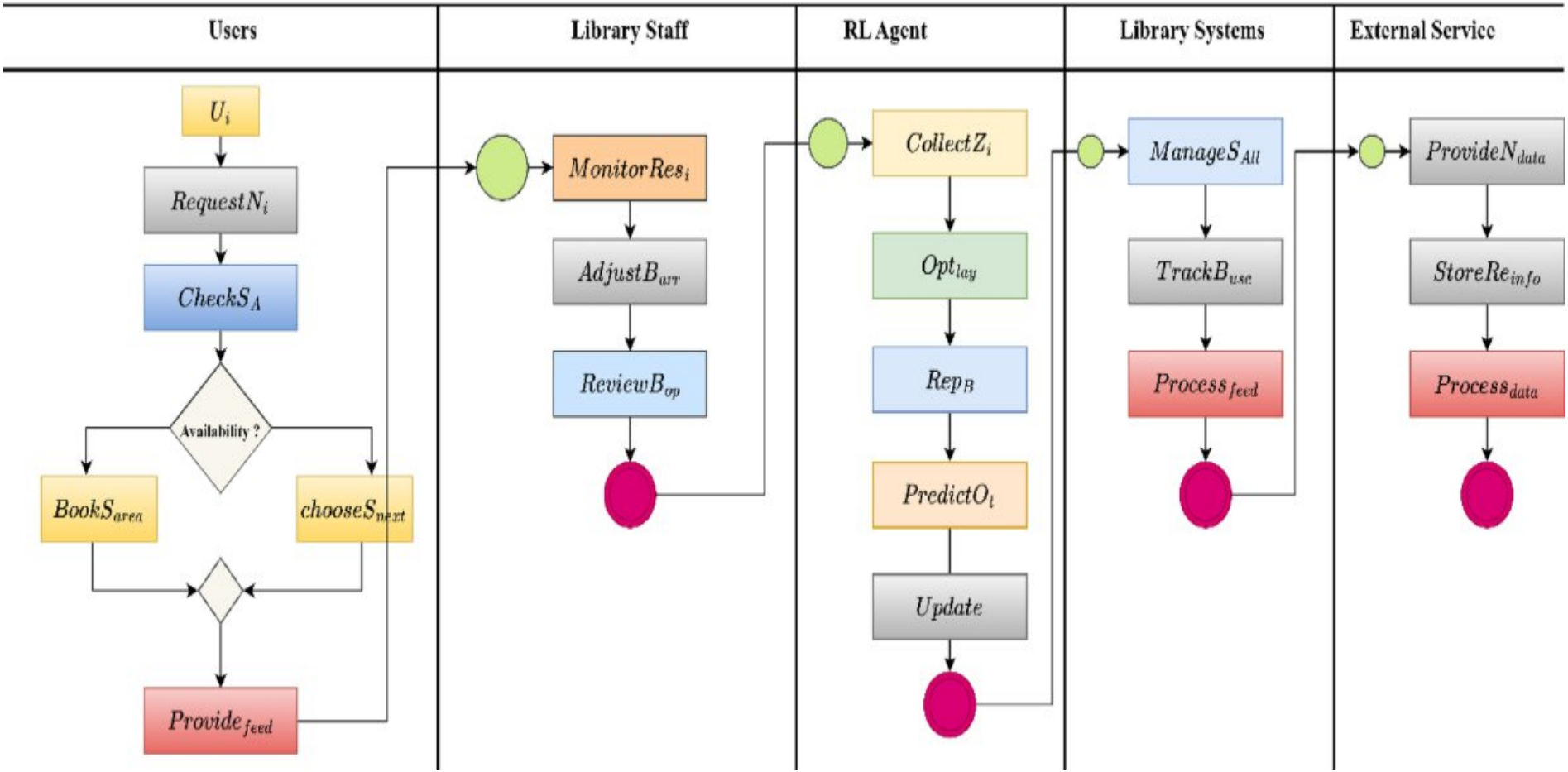
Purpose:

- To develop an automated system for efficient library request management
- To reduce manual work and minimize human errors in handling requests
- To centralize all library request data in a single system
- To streamline processes like request submission, tracking, and approval
- To improve data accuracy, security, and accessibility
- To enhance overall efficiency and productivity in library operations

Proposed Solution:

No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Managing library requests manually is time-consuming and inefficient. Librarians face issues such as misplaced requests, delayed approvals, difficulty in tracking book availability, and lack of real-time updates for users.
2.	Idea / Solution Description	The project proposes a Smart Library Request Workflow system where users can submit book requests digitally. The system automatically checks availability, routes approval to librarians, updates request status, and notifies users when books are available.
3.	Novelty / Uniqueness	• Reduces manual paperwork using automation • Digital request tracking system Real-time availability and status updates • AI-based book recommendations • Automated notifications for approvals and returns
4.	Social Impact / Customer Satisfaction	• Saves time for students and librarians • Improves accessibility to books and resources • Faster request processing • Enhances library user experience • Reduces manual errors and delays
5.	Business Model (Revenue Model)	• Can be offered as Library Management Software (SaaS) • Subscription-based model for institutions • Paid customization for schools, colleges, and organizations • Premium features like AI recommendations and analytics
6.	Scalability of the Solution	• Can handle large library databases • Easily expandable to include e-books and digital resources • Supports multiple libraries and departments • Suitable for schools, colleges, universities, and organizations

4.3 Solution Architecture



5. PROJECT PLANNING & SCHEDULING

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Before the system goes live, these planning steps ensure the "smart" infrastructure is ready to handle requests:

- **Goal Identification**: Define specific objectives like reducing manual check-outs or enhancing resource tracking.
- **Needs Assessment**: Evaluate current manual processes and feedback from staff and users.
- **Infrastructure Setup**: Deploy hardware such as **RFID tag readers**, cameras for facial recognition, and **IoT sensors**.
- **Database Configuration**: Establish master databases for **Books**, **Members**, and **Vendors**.
- **Workflow Automation Design**: Create digital triggers (e.g., via SmartSuite or Library Open Workflows) to handle approvals and notifications.

Project Tracker, Velocity & Burndown Chart:

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	9	16 Days	01 Mar 2026	15 Mar 2026	-	-
Sprint-2	7	16 Days	16 Mar 2026	01 Apr 2026	-	

Velocity:

$$V = \frac{SprintDuration}{8} = 2$$

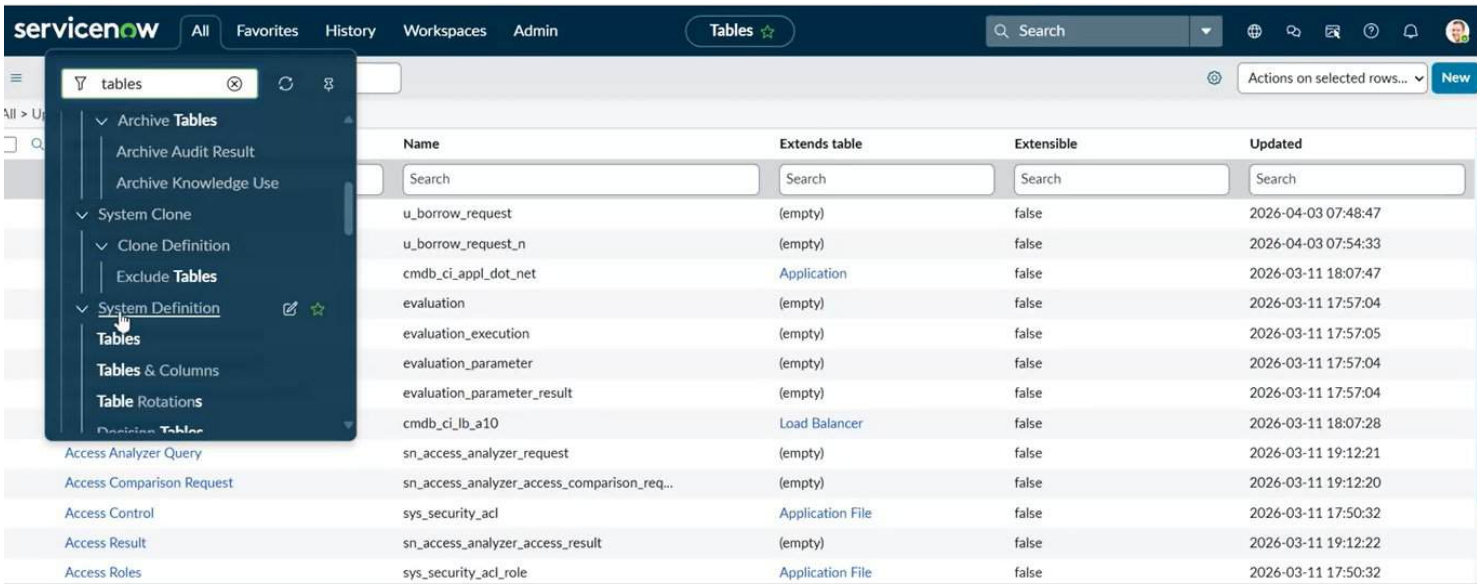
$$\therefore AV = \frac{SprintDuration}{Velocity} = \frac{16}{} = 2$$

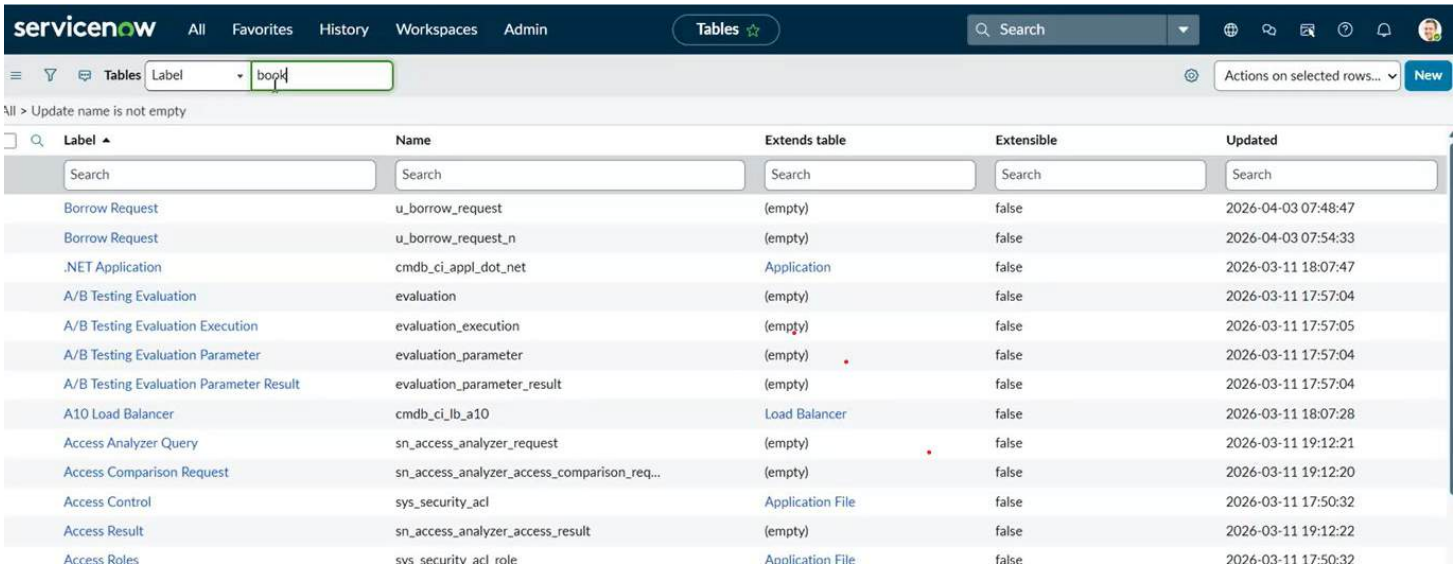
6. FUNCTIONAL AND PERFORMANCE TESTING:

Test Scenarios & Results:

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
FT-01	Book Request Submission	1. Login as user 2. Navigate to Request Book 3. Submit a request	Book request is successfully submitted and stored in the system	—	—
FT-02	User Registration	1. Open registration page 2. Enter valid details 3. Submit form	User account is created and confirmation is received	—	—
FT-03	Login Functionality	1. Enter valid username & password 2. Click login	User is logged in and redirected to dashboard	—	—
FT-04	Request Approval Workflow	1. Login as admin 2. View pending requests 3. Approve request	Request status changes to “Approved”	—	—
FT-05 Request Tracking		1. Login as user 2. Check request status	User can view real-time status (Pending/Approved/Rejected)	—	—

1. RESULTS:


servicenow
All
Favorites
History
Workspaces
Admin
Tables
Search
tables
Archive Tables
Archive Audit Result
Archive Knowledge Use
System Clone
Clone Definition
Exclude Tables
System Definition
Tables
Tables & Columns
Table Rotations
Actions on selected rows...
New
Name
Extends table
Extensible
Updated
u_borrow_request (empty) false 2026-04-03 07:48:47
u_borrow_request_n (empty) false 2026-04-03 07:54:33
cmdb_ci_appl_dot_net Application false 2026-03-11 18:07:47
evaluation (empty) false 2026-03-11 17:57:04
evaluation_execution (empty) false 2026-03-11 17:57:05
evaluation_parameter (empty) false 2026-03-11 17:57:04
evaluation_parameter_result (empty) false 2026-03-11 17:57:04
cmdb_ci_lb_a10 Load Balancer false 2026-03-11 18:07:28
Access Analyzer Query sn_access_analyzer_request (empty) false 2026-03-11 19:12:21
Access Comparison Request sn_access_analyzer_access_comparison_req... (empty) false 2026-03-11 19:12:20
Access Control sys_security_acl Application File false 2026-03-11 17:50:32
Access Result sn_access_analyzer_access_result (empty) false 2026-03-11 19:12:22
Access Roles sys_security_acl_role Application File false 2026-03-11 17:50:32


servicenow
All
Favorites
History
Workspaces
Admin
Tables
Search
Tables
Label
book
Actions on selected rows...
New
All > Update name is not empty
Label
Name
Extends table
Extensible
Updated
Borrow Request u_borrow_request (empty) false 2026-04-03 07:48:47
Borrow Request u_borrow_request_n (empty) false 2026-04-03 07:54:33
.NET Application cmdb_ci_appl_dot_net Application false 2026-03-11 18:07:47
A/B Testing Evaluation evaluation (empty) false 2026-03-11 17:57:04
A/B Testing Evaluation Execution evaluation_execution (empty) false 2026-03-11 17:57:05
A/B Testing Evaluation Parameter evaluation_parameter (empty) false 2026-03-11 17:57:04
A/B Testing Evaluation Parameter Result evaluation_parameter_result (empty) false 2026-03-11 17:57:04
A10 Load Balancer cmdb_ci_lb_a10 Load Balancer false 2026-03-11 18:07:28
Access Analyzer Query sn_access_analyzer_request (empty) false 2026-03-11 19:12:21
Access Comparison Request sn_access_analyzer_access_comparison_req... (empty) false 2026-03-11 19:12:20
Access Control sys_security_acl Application File false 2026-03-11 17:50:32
Access Result sn_access_analyzer_access_result (empty) false 2026-03-11 19:12:22

servicenow All Favorites History Workspaces Admin Tables Search

Tables Label Search Actions on selected rows... New

All > Update name is not empty > Label >= book

Label	Name	Extends table	Extensible	Updated
Book	u_book	(empty)	false	2026-04-03 07:47:46
Book [u_book]	u_book_u_book_	(empty)	false	2026-04-03 14:21:51
Bookmark	sys_ui_bookmark	(empty)	false	2026-03-11 17:55:09
Bookmark Group	sys_ui_bookmark_group	(empty)	false	2026-03-11 17:55:09
Bot Messages	sys_cs_bot_messages	(empty)	false	2026-03-11 19:04:41
Bounce Email Address	sys_blocked_email_address	(empty)	false	2026-03-11 17:51:44
Bounce Email Address Log	sys_blocked_email_address_log	(empty)	false	2026-03-11 17:51:44
Bounce Email Address Status	sys_bounce_email_address_status	(empty)	false	2026-03-11 17:51:44
Boundary Connection Qualifier	cmdb_ci_qualifier_boundary_connection	Qualifier	false	2026-03-11 18:06:22
Branding	sys_cs_branding_setup	Application File	false	2026-03-11 19:04:08
Breakdown	pa_breakdowns	Application File	false	2026-03-11 18:30:18
Breakdown	sn_bm_client_breakdown	(empty)	false	2026-03-11 18:36:36

servicenow All Favorites History Workspaces Admin Table - Book Search

Table Book Delete Update Delete All Records

A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. More Info

* Label Book
* Name u_book
Application Global
Remote Table

Columns Controls Application Access

Table Columns for text Search 1 to 6 of 6 New

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Updates	Integer	(empty)	40		false
Created by	String	(empty)	40		false
Updated by	String	(empty)	40		false
Sys ID	Sys ID (GUID)	(empty)	32		false

servicenow All Favorites History Workspaces Admin Table - Borrow Request Search

Table
Borrow Request

Delete Update Delete All Records

Updated by	String	(empty)	40	false
Updates	Integer	(empty)	40	false
Updated	Date/Time	(empty)	40	false
Created by	String	(empty)	40	false
Book	Reference	Book [u_book]	32	false
Insert a new row...				

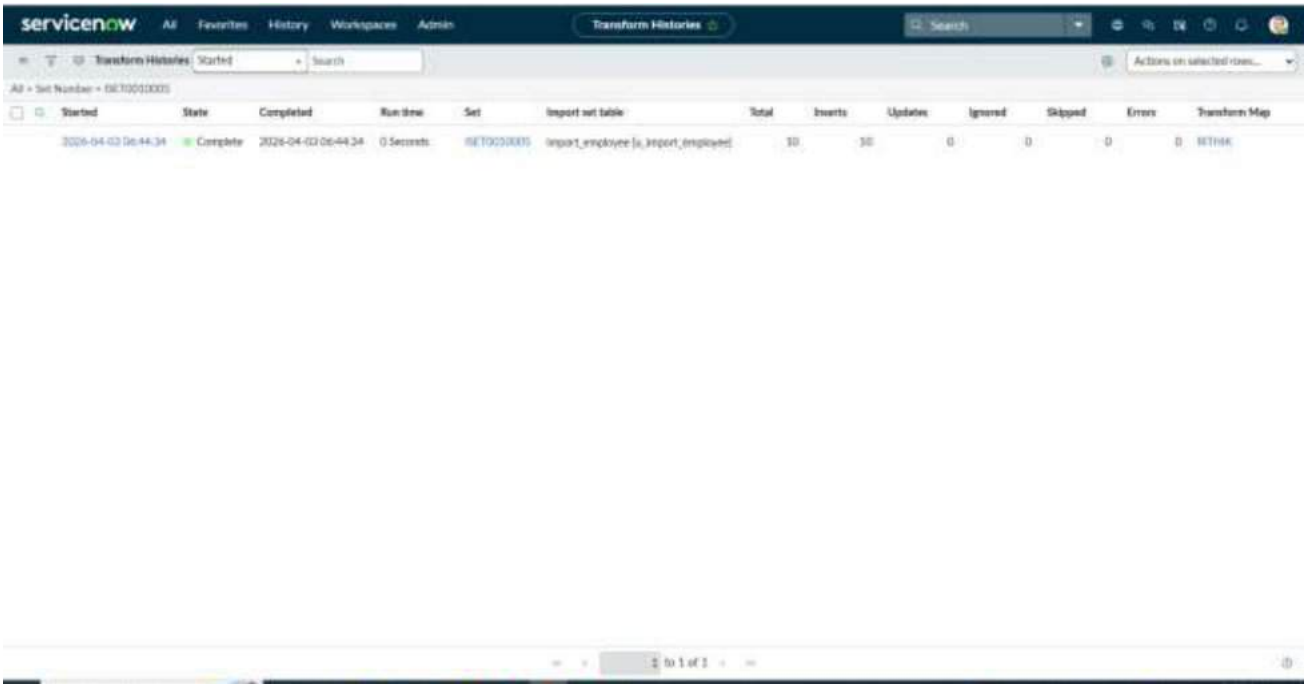
Delete Update Delete All Records

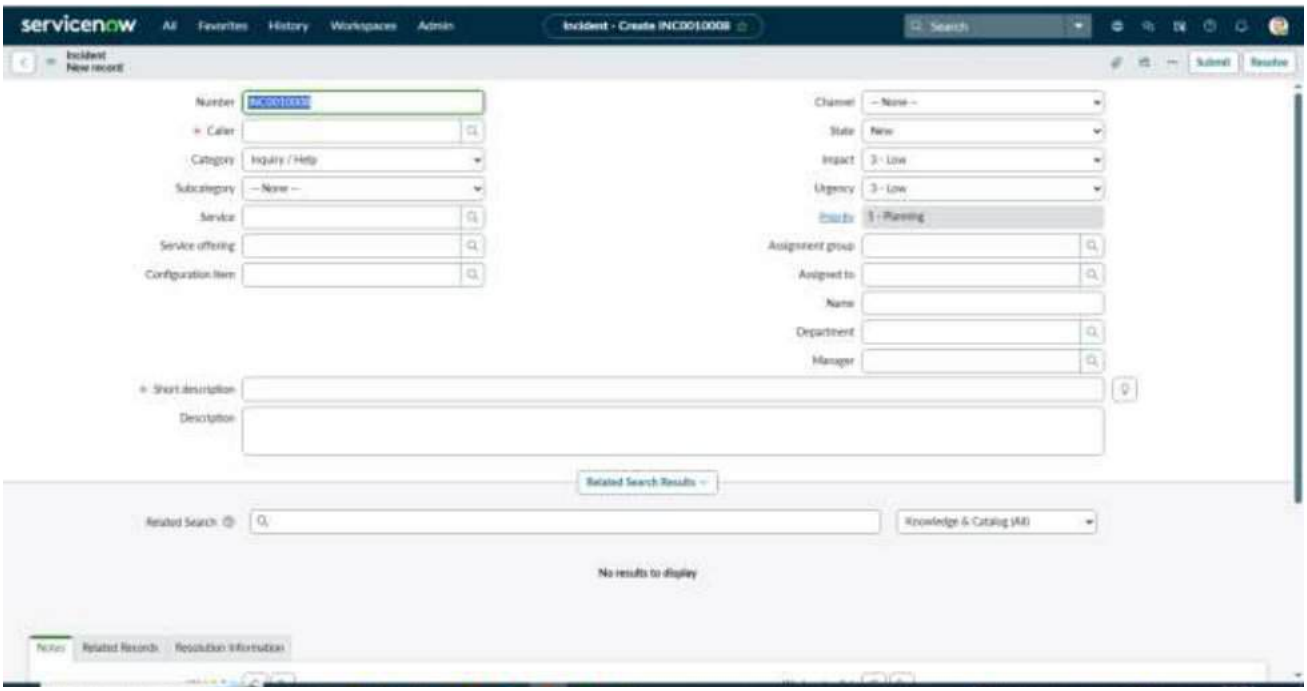
Related Links
Form Builder
Design Form
Layout Form
Layout List
Show Form
Show List
Show Schema Map
Add to Service Catalog
Run Point Scan
Explore REST API

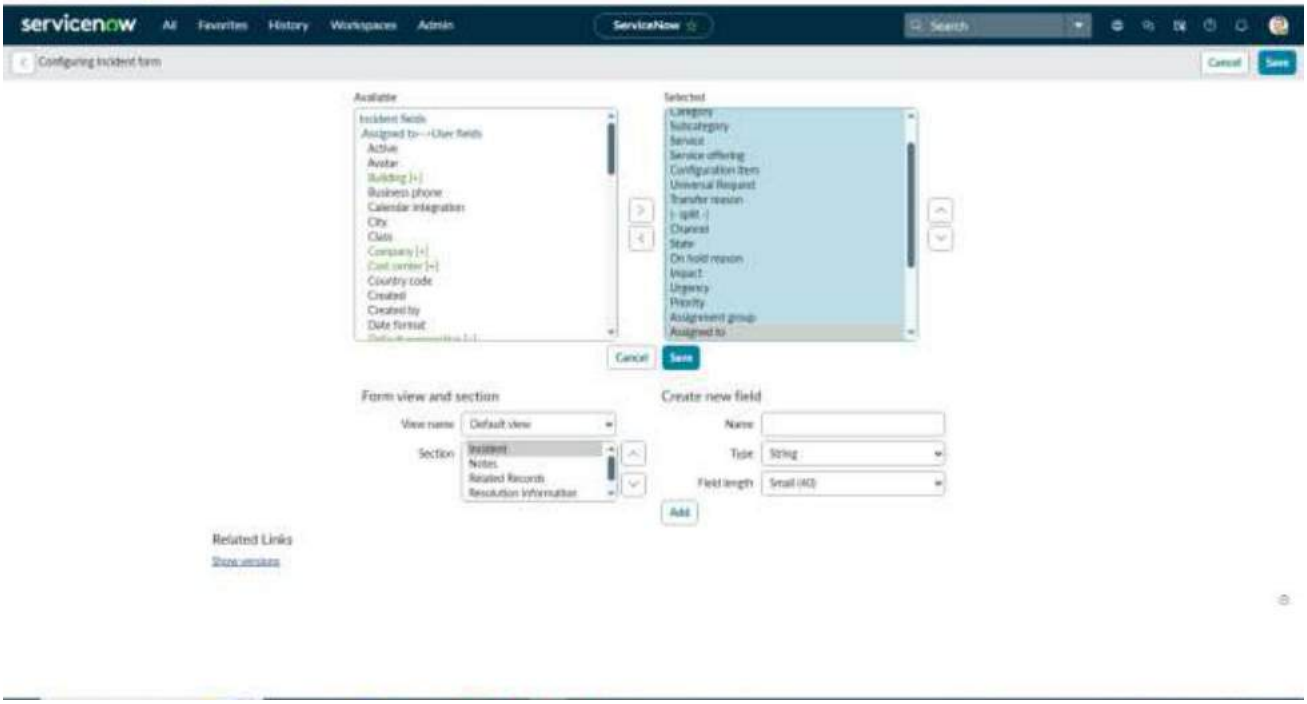
servicenow All Favorites History Workspaces Admin Report Sources Search

Report Sources Name Search Actions on selected rows... New

Name	Table	Filter
Search	Search	Search
Change requests.Closed	Change Request [change_request]	Active = false
Change requests.Open	Change Request [change_request]	Active = true
Change requests.Open.Emergency	Change Request [change_request]	Active = true .and. Type = Emergency
Contracts.Active	Contract [ast_contract]	Active = true
Contracts.Active.Expiring	Contract [ast_contract]	State = Active .and. End date < javascri...
Feedback.Comments	Knowledge Feedback [kb_feedback]	Comments is not empty
Feedback.Flagged	Knowledge Feedback [kb_feedback]	Flagged = true
Feedback.Not Useful	Knowledge Feedback [kb_feedback]	Useful = No


servicenow
All
Favorites
History
Workspaces
Admin
Transform Histories
Transform Histories
Started
Actions on selected rows...
All > Set Number = ISET0010001
Started
State
Completed
Run time
Set
Import set table
Total
Inserts
Updates
Ignored
Skipped
Errors
Transform Map
2026-04-03 06:44:34
Complete
2026-04-03 06:44:34
0 Seconds
ISET0010001
u_import_employee [u_import_employee]
10
10
0
0
0
0
NTHK
1 to 1 of 1


servicenow
All
Favorites
History
Workspaces
Admin
Incident - Create INC0010008
Search
Incident
New record
Submit
Resolve
Number
INC0010008
Caller
Category
Inquiry / Help
Subcategory
-- None --
Service
Service offering
Configuration item
Channel
-- None --
State
New
Impact
3 - Low
Urgency
3 - Low
Priority
5 - Planning
Assignment group
Assigned to
Name
Department
Manager
Short description
Description
Related Search Results
Related Search
Knowledge & Catalog (All)
No results to display
Notes
Related Records
Resolution Information


servicenow
All
Favorites
History
Workspaces
Admin
ServiceNow
Search
Configuring Incident form
Cancel
Save
Available
Incident fields
Assigned to---User fields
Active
Avatar
Building [+]
Business phone
Calendar integration
City
Class
Company [+]
Cost center [+]
Country code
Created
Created by
Date format
Selected
Category
Subcategory
Service
Service offering
Configuration item
Universal Request
Transfer reason
[- split -]
Channel
State
On hold reason
Impact
Urgency
Priority
Assignment group
Assigned to
Cancel
Save
Form view and section
View name
Default view
Section
Incident
Notes
Related Records
Resolution Information
Create new field
Name
Type
String
Field length
Small (40)
Add
Related Links
Show versions

8. ADVANTAGES & DISADVANTAGES

8.1. Advantages:

1. **Significant Time Savings (Reduced Processing Time):**
   By automating the book request and approval workflow, users and librarians no longer need to manually track requests. This reduces the time required to process and approve book requests efficiently.
2. **Elimination of Manual Effort:**
   The system automates request submission, tracking, and approval, eliminating the need for manual record-keeping and reducing human errors.
3. **Highly Maintainable (Low-Code/No-Code):**
   The workflow is built using standard platform features such as forms, workflows, and configurations. This makes the system easy to maintain without requiring complex coding.
4. **Upgrade Safe:**
   Since the solution uses native platform capabilities, it remains stable and compatible during system upgrades, ensuring uninterrupted functionality.
5. **Scalability:**
   The system can handle a large number of book requests efficiently, whether it is a small library or a large institution with thousands of users.

8.2. Disadvantages & Limitations:

1. **Dependence on Data Accuracy:**
   The system relies on correct input from users. If users enter incorrect book details or wrong information, it may lead to errors in request processing or approvals.
2. **Not Fully Real-Time in Some Cases:**
   If notifications or updates are processed in batches (e.g., scheduled jobs), there may be slight delays in reflecting request status or approvals.
3. **Possible UI Complexity:**
   Adding too many fields (like book details, user info, request status, history, etc.) can make the interface cluttered and difficult for users to navigate.
4. **Limited Direct Editing:**
   Some workflow data (like approved requests or system-generated details) may be read-only. Users or librarians may need to navigate to other sections to make corrections.
5. **Initial Setup Complexity:**
   Configuring workflows, roles, permissions, and request flows requires a good understanding of the platform. Incorrect setup may lead to workflow errors or access issues.

9. CONCLUSION:

The **Smart Library Request Workflow** project has successfully improved the efficiency and management of library operations by automating the book request and approval process. By replacing manual request handling with a structured digital workflow, the system ensures accurate tracking, faster processing, and reduced human errors.

The implementation of automated workflows enables users to easily submit book requests, while librarians can efficiently review, approve, or reject them. This streamlined process provides real-time visibility of request status, improving transparency and user satisfaction.

Additionally, the use of native platform features ensures that the system is highly scalable, maintainable, and secure. The low-code approach minimizes complexity and allows easy future enhancements without major changes.

Overall, this project demonstrates how automation can significantly enhance library management by reducing workload, improving response time, and creating a more organized and user-friendly system. It also provides a strong foundation for further improvements such as notifications, analytics, and integration with other systems.

10. FUTURE SCOPE:

1. **Real-Time Integration (Eliminating Manual Entry):**
   Future improvements can include integrating the system with external library databases or APIs. This will allow real-time updates of book availability, eliminating the need for manual data entry.
2. **Expansion Across Modules:**
   Currently, the workflow is focused on book requests. In future, it can be extended to other modules such as book returns, renewals, reservations, and fine management, providing a complete library management solution.
3. **Automated Allocation & Recommendations:**
   The system can be enhanced to automatically assign books based on availability and even recommend books to users based on their reading history or preferences.
4. **Smart Notifications & Alerts:**
   Future versions can include automated notifications such as request approval alerts, due date reminders, overdue warnings, and availability notifications for requested books.
5. **Advanced Validation & Error Handling:**
   To improve data accuracy, validation mechanisms can be added to verify user inputs and prevent incorrect or duplicate requests. Invalid entries can be flagged for review before processing.